ANDROID APP DEVELOPMENT USING GEN AI INTERNSHIP PROGRAM

Layout Planning & Review

Business Card App

Layout Planning • Alignment • Accessibility • Performance Review

Student Name:	Guru K.R.
Course:	Internship Program
Report Type:	Layout Planning & Review Submission
App Name:	Business Card App (com.example.businesscard)
Submission Date:	February 2026

1. Introduction

The Business Card App is a single-screen Android application developed using Jetpack Compose that presents a user's professional contact information in a clean, visually structured card format. The screen displays a profile image or logo, the user's full name, job title, and three contact detail rows comprising a phone number, social media handle, and email address. The primary objective of this application is to demonstrate the effective use of Compose layout composables — particularly Column, Image, Text, and Row — to produce a polished, readable, and accessible UI without relying on XML layout files.

Layout planning is a critical precursor to coding in Compose development. Defining the composable hierarchy, spacing values, font sizes, alignment strategies, and accessibility requirements before implementation begins ensures that design decisions are intentional rather than reactive. This planning process reduces structural refactoring during development, improves visual consistency in the final output, and establishes a clear reference document against which the implemented screen can be evaluated. This report documents the layout plan, composable structure, and design review for the Business Card App.

2. Screen Overview

The Business Card App screen is organised as a single vertically stacked layout, drawing the user's eye downward from a prominent logo or profile image at the top, through the name and role text, and finally to the contact information rows at the bottom of the screen. The overall layout uses a light green background colour (0xFFD2E8D4) to create a calm, professional aesthetic that contrasts clearly with the dark text and green accent typography.

2.1 Screen Element Breakdown

- **Profile Image / Logo (Top):** The Android logo image is displayed at the top centre of the screen. It acts as the visual anchor for the card, establishing the user's brand identity immediately.
- **Name (Large Text):** The user's name — "Guru" — is displayed in a large, bold typeface centred below the image. This is the most prominent text element on the screen.
- **Job Title (Medium Text):** The role descriptor "Android Developer Extraordinaire" appears in a green accent colour below the name, styled at a medium font size to establish clear typographic hierarchy between the name and the role.
- **Contact Information Rows:** Three contact detail rows are displayed in the lower portion of the screen, each comprising a small icon on the left and a text label on the right. The three rows cover phone number (+91 9876543210), social handle (@GuruDev), and email address (guru@email.com).
- **Structured Vertical Alignment:** All elements are centred horizontally within a Column container that spans the full screen. Contact rows use start alignment internally to align the icon and text consistently from the left edge.

2.2 Implementation Screenshots

The following screenshots document the Business Card App as implemented in Android Studio, including the Kotlin source code and the rendered output on the Pixel 6 API 36 emulator.

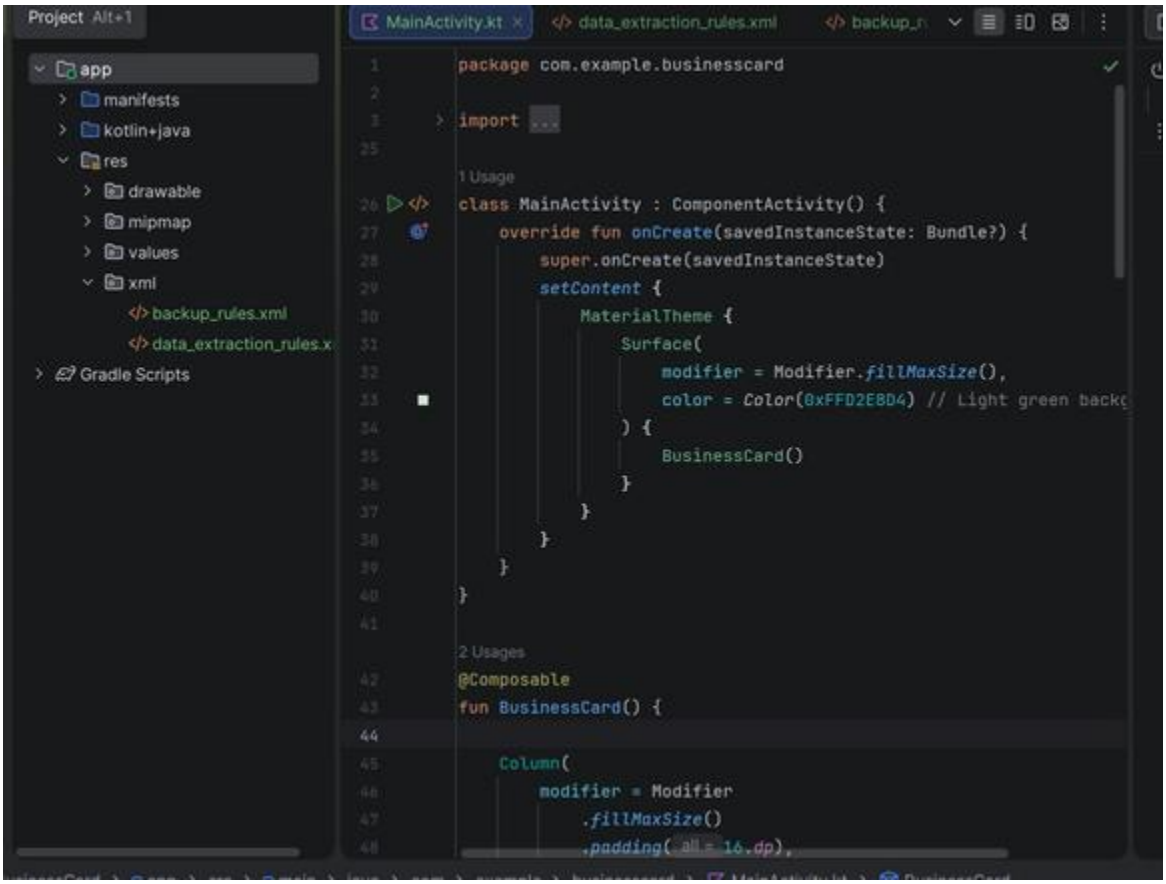

Figure 1 — Android Studio: MainActivity.kt showing MaterialTheme, Surface with light green background (0xFFD2E8D4), and BusinessCard() composable call

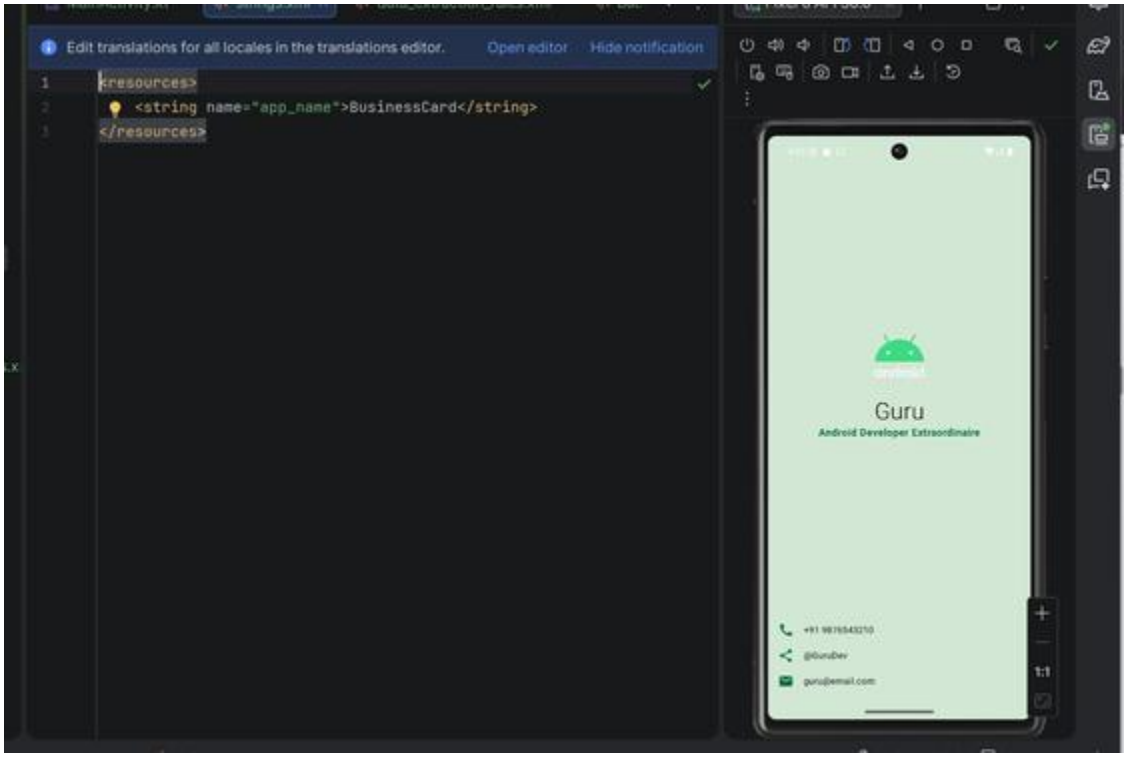

Figure 2 — Android Studio: strings.xml resource file alongside the Pixel 6 emulator rendering the Business Card layout — profile image, name, role, and contact rows visible

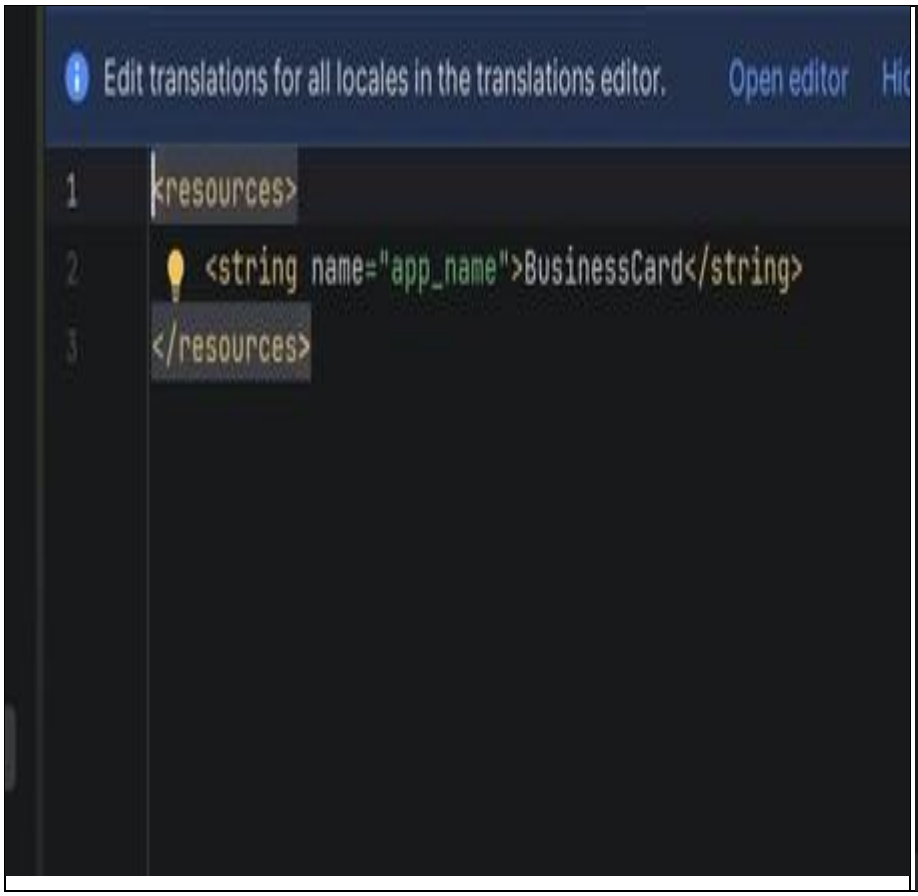

Figure 3 — strings.xml: app_name resource set to "BusinessCard" | Figure 4 — Pixel 6 emulator close-up: rendered Business Card App showing Android logo, Guru, Android Developer Extraordinaire, and contact detail rows

2.3 Layout Wireframe

The wireframe below illustrates the planned layout structure of the Business Card App screen prior to implementation. Each UI element is annotated with the composable used and the spacing applied.

Figure 5 — Business Card App Layout Wireframe: composable annotations and spacing values

3. Composable Structure Plan

The Business Card App UI is constructed from a carefully chosen set of Jetpack Compose composables. Each composable was selected to fulfil a specific structural or presentational role within the layout hierarchy. The following subsections describe each composable, its role, and the rationale behind its selection and configuration.

3.1 Column — Main Vertical Layout Container

The Column composable serves as the root layout container for the entire Business Card screen. It arranges all child elements — the logo image, name text, role text, and contact rows — in a vertical sequence from top to bottom. The Column is configured with horizontalAlignment = Alignment.CenterHorizontally, ensuring that all children are automatically centred along the horizontal axis without requiring individual alignment modifiers on each child element. A Modifier.fillMaxSize() is applied so that the Column occupies the entire screen surface, and a uniform padding of 16dp on all sides ensures that no content touches the device screen edges.

The decision to use a single root Column rather than nested layout containers was deliberate. A flat hierarchy reduces the composable tree depth, which in turn improves Compose's recomposition efficiency and simplifies layout debugging.

3.2 Image — Profile Logo

The Image composable renders the Android logo at the top of the card layout. The image is loaded from the local drawable resource folder using a painterResource reference. A fixed size modifier is applied to constrain the image to a square display area (approximately 120 × 120dp), preventing the image from consuming disproportionate vertical space on smaller screens. ContentScale.Fit is applied to ensure the image scales proportionally within its bounds without distortion or cropping. A contentDescription of "Android logo" is applied for screen reader accessibility compliance.

3.3 Text — Name, Role, and Contact Labels

Three distinct Text composables are used for the primary text content of the card:

- **Name Text:** Displays the user's name "Guru" at a font size of 36sp with bold weight. This is the largest text element on the screen, ensuring the name is the first item read by the user after the logo image. Centred horizontal alignment is inherited from the parent Column.
- **Role Text:** Displays "Android Developer Extraordinaire" at 18sp, styled in a green accent colour to differentiate it visually from the name without requiring a size-only hierarchy distinction. The colour choice also reinforces the app's green-accented visual identity.
- **Contact Detail Text:** Each contact detail row contains a Text composable displaying the contact value (phone number, social handle, or email address) at 16sp in a standard dark body colour. The consistent font size across all three contact rows creates visual uniformity in the lower section of the card.

3.4 Row — Icon and Contact Text Alignment

Each of the three contact detail entries is implemented as a Row composable containing an Icon composable on the left and a Text composable on the right. The Row arrangement ensures the icon and its associated text remain horizontally aligned regardless of text length variations. The Row uses a verticalAlignment = Alignment.CenterVertically parameter so that the icon and text baseline are visually balanced even if the icon height differs slightly from the text height. A spacing of 8dp is applied between the icon and text within each Row, and 8dp vertical spacing separates each contact Row from the next.

3.5 Modifier.padding and Spacing Decisions

Location	Value	Rationale
Column outer padding (all sides)	16dp	Prevents content from touching screen edges on any device
Image → Name Text	16dp	Separates the logo from the name; establishes clear vertical rhythm
Name Text → Role Text	8dp	Close grouping signals name and role belong together

Role Text → Contact section	24dp	Larger gap creates visual separation between identity and contact info
Between contact Rows	8dp	Uniform spacing makes the contact list scannable and consistent
Icon ↔ Text within Row	8dp	Sufficient separation for legibility without wasted horizontal space

4. Alignment & Readability Review

4.1 Font Size Hierarchy

The Business Card App employs a three-tier typographic hierarchy to guide the reader's eye through the card content in the intended order of importance:

Element	Font Size	Weight	Role in Hierarchy
Name (Guru)	36sp	Bold	Primary — largest, first visual read after logo
Role (Job Title)	18sp	Normal	Secondary — differentiates by colour (green accent)
Contact Details	16sp	Normal	Tertiary — uniform, scannable information list

4.2 Consistent Spacing

Spacing consistency is maintained across all vertical gaps in the layout by using multiples of the base 8dp grid. The outer padding is 16dp (2 × 8dp), the standard element gap is 8dp, and the separator between the identity section and contact section is 24dp (3 × 8dp). Adherence to an 8dp grid prevents irregular visual gaps that create a sense of imbalance and makes the layout appear professionally designed.

4.3 Text Alignment

The name and role Text composables use centred horizontal alignment, inherited from the Column's horizontalAlignment = Alignment.CenterHorizontally parameter. This centred alignment is appropriate for a card UI where the name and role function as a visual identity block rather than a continuous reading passage. Centred text for short strings (a name and a job title) creates symmetry and visual balance.

The contact detail Rows, by contrast, use start alignment for the Row's children (icon followed by text), with the Row itself centred within the Column. This mixed approach — centred container, left-aligned content — is a deliberate readability decision: contact details are read left to right and benefit from consistent start alignment, while the Row as a unit remains visually balanced within the centred card layout.

4.4 Visual Balance

The overall visual balance of the Business Card screen is strong. The logo image at the top provides a focal point, the large name text immediately below it creates a clear visual anchor, and the three uniformly styled contact rows at the bottom provide a structured, predictable close to the card. The green accent colour on the role text and contact icons ties the visual identity together without introducing colour complexity. The light green background ensures sufficient contrast for all dark text elements.

5. Responsiveness Considerations

5.1 Behaviour on Small vs Large Screens

On standard phone screens (360–412dp width), the Business Card layout performs well. The Column with fillMaxSize() ensures the layout expands to occupy the full screen, and the 16dp outer padding prevents content from touching device edges. The fixed image size and text sizes are appropriate for phone-sized screens and do not cause content to overflow vertically on devices with screen heights of 640dp or greater.

On small screens (below 360dp width or below 640dp height), the layout may experience slight compression. If the image size is fixed at a large value (e.g., 140dp), the combined height of the image, name, role, spacing, and three contact rows may approach the lower boundary of available vertical space, potentially requiring the user to scroll if the content height exceeds the screen height. On large screens and tablets (600dp+ width), the full-width Column may appear visually sparse, with excessive white space on both sides of the card content.

5.2 Content Overflow Risks

The role text field "Android Developer Extraordinaire" is a relatively long string. On very narrow screens (below 320dp), this text may wrap to two lines, increasing the overall vertical height of the layout. Similarly, long email addresses or phone numbers in the contact rows could overflow horizontally on narrow displays if the Row does not apply text overflow ellipsis behaviour. These overflow risks should be mitigated by applying maxLines = 1 and overflow = TextOverflow.Ellipsis to contact Text composables.

5.3 Improvements for Tablets

On tablet screens (600dp+), a more appropriate layout would place the profile image on the left side of a Row composable, with the name, role, and contact information stacked in a Column on the right. This side-by-side arrangement uses the available horizontal space more effectively than a centred vertical stack, producing a layout that resembles a physical business card in landscape orientation. Adaptive layout logic using LocalConfiguration or WindowSizeClass APIs would allow the app to switch between the phone (Column) and tablet (Row) layouts automatically based on the available screen width.

6. Accessibility Review

6.1 Text Legibility

The font sizes used in the Business Card App — 36sp for the name, 18sp for the role, and 16sp for contact details — exceed the minimum recommended sizes for readability on mobile screens. The dark text colours used against the light green background (0xFFD2E8D4) provide adequate contrast for standard vision users. However, the green accent colour used for the role text and contact icons should be verified against the WCAG AA standard (minimum 4.5:1 contrast ratio for normal text) to ensure legibility for users with colour vision deficiencies.

6.2 Touch Target Sizing

The Business Card App in its current form is primarily a display screen with no interactive touch elements beyond the potential for tappable contact rows (e.g., tapping the phone number to initiate a call). If contact rows are made interactive in future iterations, each Row must meet the Material Design minimum touch target size of 48dp height. The current 8dp spacing between rows provides sufficient separation to prevent accidental activation of adjacent rows, but the Row height itself must be verified to meet the 48dp minimum when interactive behaviour is added.

6.3 Content Descriptions for Images and Icons

The Android logo Image composable must carry a contentDescription parameter set to a descriptive string such as "Android logo" to enable TalkBack and other screen reader tools to communicate the image's purpose to visually impaired users. Without a contentDescription, the image is either ignored by screen readers or announced as "unlabelled image", which provides no meaningful information.

Similarly, each Icon composable within the contact detail Rows should carry a contentDescription describing the icon's meaning (e.g., "Phone number", "Social media handle", "Email address"). Alternatively, if the icon is considered purely decorative — because the adjacent Text already provides the full information — the contentDescription can be set to null to suppress the screen reader announcement and avoid redundant narration.

6.4 Spacing Between Interactive Elements

If the contact rows are made tappable in future, the 8dp vertical gap between rows may be insufficient to prevent accidental touch activation on high-sensitivity touch screens. Material Design guidelines recommend at minimum 8dp of visual separation between interactive targets, with the touch target height being at least 48dp per element. The current layout should be reviewed to ensure compliance when interactive behaviour is introduced.

7. Performance Considerations

7.1 Optimised Image Size

The Android logo image used as the profile picture in the Business Card App should be stored as an optimised PNG or SVG file in the drawable resource folder. A PNG file for a logo displayed at 120×120dp on a standard MDPI screen need not exceed 240×240 pixels (accounting for up to 2× density scaling). Storing an unnecessarily large image (e.g., 1000×1000px or higher) increases the APK binary size and requires more memory to decode and render at runtime, introducing potential lag on devices with limited RAM.

The res/drawable folder structure observed in the project (as shown in Figure 1) does not utilise density-specific sub-folders (e.g., drawable-hdpi, drawable-xhdpi). For raster images, using density-appropriate folders or, preferably, vector drawable assets ensures the image scales correctly across all device densities without manual resolution management.

7.2 Avoiding Unnecessary Nested Layouts

The Business Card App's composable structure uses a flat layout hierarchy: a single Column at the root, with Image, Text, and Row composables as direct children. This structure avoids deeply nested layouts (e.g., Column within Column within Box within Column), which can increase the complexity of Compose's layout measurement phase and introduce unnecessary recomposition overhead. Each Row within the contact section contains only two children — an Icon and a Text — maintaining minimal nesting depth throughout the tree.

7.3 Efficient Composable Usage

The Business Card screen is a static display screen with no state changes after initial composition. Because none of the composables in the tree observe any mutable State objects, the screen undergoes a single initial composition pass and does not trigger recomposition during normal viewing. This makes the screen extremely efficient from a recomposition perspective. However, if future iterations add interactive features (e.g., tappable contact links that open dialler or email apps), state management should be introduced carefully to avoid triggering unnecessary recomposition of unrelated composable subtrees.

8. Improvement Plan — Mini Action Plan

Based on the alignment, responsiveness, accessibility, and performance reviews conducted in this report, the following concrete improvements are recommended for implementation before the final release of the Business Card App.

#	Improvement	Details	Phase	Priority
1	Add contentDescription to all images and icons	Set contentDescription="Android logo" on the Image composable; set meaningful or null contentDescription on each contact Row Icon based on whether the adjacent Text provides full context.	Before Coding	High
2	Apply TextOverflow.Ellipsis to contact Text elements	Add maxLines=1 and overflow=TextOverflow.Ellipsis to all contact detail Text composables to prevent horizontal overflow on narrow screens and very long strings.	During Coding	Medium
3	Verify green accent colour contrast ratio (WCAG AA)	Measure the contrast ratio of the green accent colour (#4CAF50) against the light green background (0xFFD2E8D4) using a WCAG-compliant contrast checker. Adjust the green shade if the ratio falls below 4.5:1 for normal text.	Before Coding	High
4	Implement adaptive tablet layout using Row	Introduce a WindowSizeClass check to switch from the current Column-only layout to a Row-based layout (image on left, text+contacts on right) for screens 600dp wide or greater, improving the tablet experience.	After Testing	Low

9. Reflection

Completing this layout planning and review exercise for the Business Card App reinforced the value of approaching UI development as a structured design process rather than an improvisational coding activity. By mapping the composable hierarchy — Column as root, Image at top, Text for identity elements, Row for contact entries — before writing any implementation code, the resulting structure in MainActivity.kt directly reflected the intended layout without requiring mid-implementation refactoring. The screenshots captured in Figures 1 through 4 confirm that the planned composable structure translated faithfully into the rendered output on the Pixel 6 emulator, validating the planning process.

The accessibility review, in particular, demonstrated that concerns such as missing contentDescription attributes and unverified colour contrast ratios are far more efficiently addressed at the design stage than after the screen has been implemented and tested. Retrofitting accessibility features into an existing codebase requires code changes in multiple locations; planning for them from the outset means they are built into the first implementation. Similarly, the responsiveness review identified tablet layout improvements that, if planned early, would inform the initial composable structure rather than requiring a separate adaptive layout refactor at a later stage.

The performance considerations discussed in Section 7 — flat composable hierarchy, optimised image assets, and static composition — were all natural outcomes of the planning decisions made before coding. A flat Column-based layout is both the simplest and the most performant structure for a single-screen card display, and this alignment between planning intent and performance outcome illustrates that good layout design and efficient Compose code are

not competing objectives but complementary ones. This internship exercise has strengthened the discipline of planning before coding as a core professional practice in Android development.